

# PE-EC702C — Neural Network & Fuzzy Logic Control

Semester VII • 3L:0T:0P • 3 Credits

## Module 3 Exam Notes: Adaptive Resonance & Backpropagation

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Backpropagation Neural Networks (BPNN)</b> . . . . .    | <b>2</b> |
| 1.1      | Derivation of the Backpropagation Learning Rules . . . . . | 2        |
| <b>2</b> | <b>Adaptive Resonance Theory (ART)</b> . . . . .           | <b>3</b> |
| 2.1      | ART1 vs. ART2 . . . . .                                    | 3        |
| 2.2      | ART1 Architecture and Operation . . . . .                  | 3        |
| <b>3</b> | <b>Boltzmann Machine Learning</b> . . . . .                | <b>4</b> |
| 3.1      | Boltzmann Learning Rule . . . . .                          | 5        |
| <b>4</b> | <b>Worked Examples</b> . . . . .                           | <b>5</b> |

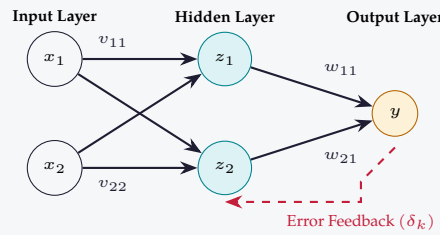
## 1. Backpropagation Neural Networks (BPNN)

The Backpropagation algorithm is the standard method for training Multi-Layer Perceptrons (MLPs). It is a supervised learning method based on the generalized delta rule (gradient descent on error space).

### Multi-Layer Perceptron (MLP)

A **Multi-Layer Perceptron (MLP)** is a feedforward artificial neural network topology consisting of an input layer, one or more hidden layers, and an output layer of non-linear processing units.

### Multi-Layer Perceptron Feedforward Architecture



### 1.1 Derivation of the Backpropagation Learning Rules

Let  $E$  be the total sum-of-squares error over the output units:

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2 \quad (1)$$

where  $t_k$  is the target output and  $y_k$  is the computed activation of output unit  $k$ . The activation function used is the binary sigmoid:

$$y_k = f(net_k) = \frac{1}{1 + e^{-net_k}} \rightarrow f'(net_k) = y_k(1 - y_k) \quad (2)$$

where  $net_k = \sum_j w_{kj} z_j + b_k$  ( $z_j$  is the activation of hidden node  $j$ ).

Using gradient descent, the weight changes are proportional to the negative gradient of error:

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}} \quad (3)$$

By applying the chain rule of calculus:

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial net_k} \cdot \frac{\partial net_k}{\partial w_{kj}} \quad (4)$$

Evaluating each derivative:

$$\frac{\partial E}{\partial y_k} = -(t_k - y_k), \quad \frac{\partial y_k}{\partial net_k} = f'(net_k) = y_k(1 - y_k), \quad \frac{\partial net_k}{\partial w_{kj}} = z_j$$

Substituting these terms back:

$$\frac{\partial E}{\partial w_{kj}} = -(t_k - y_k) f'(net_k) z_j \quad (5)$$

Defining the error term (delta) for output unit  $k$  as  $\delta_k = (t_k - y_k)f'(net_k)$ :

$$\Delta w_{kj} = \eta \delta_k z_j \quad (6)$$

For input-to-hidden weights  $v_{ji}$  connecting input  $i$  to hidden unit  $j$ :

$$\Delta v_{ji} = -\eta \frac{\partial E}{\partial v_{ji}} = -\eta \frac{\partial E}{\partial z_j} \cdot \frac{\partial z_j}{\partial net_j} \cdot \frac{\partial net_j}{\partial v_{ji}} \quad (7)$$

where  $z_j = f(net_j)$  and  $net_j = \sum_i v_{ji}x_i + b_j$ . The error gradient relative to hidden activation  $z_j$  accumulates from all output nodes:

$$\frac{\partial E}{\partial z_j} = \sum_k \frac{\partial E}{\partial net_k} \cdot \frac{\partial net_k}{\partial z_j} = \sum_k (-\delta_k) w_{kj}$$

Applying the other terms:

$$\frac{\partial z_j}{\partial net_j} = f'(net_j) = z_j(1 - z_j), \quad \frac{\partial net_j}{\partial v_{ji}} = x_i$$

Defining the error term (delta) for hidden unit  $j$  as  $\delta_j = f'(net_j) \sum_k \delta_k w_{kj}$ :

$$\Delta v_{ji} = \eta \delta_j x_i \quad (8)$$

## 2. Adaptive Resonance Theory (ART)

Developed by Stephen Grossberg and Gail Carpenter (1987), ART is designed to solve the stability-plasticity dilemma in unsupervised neural networks.

### Stability-Plasticity Dilemma

**Stability-Plasticity Dilemma:** The challenge of designing a learning system that remains adaptive to new patterns (plasticity) while preventing new learning from washing out previously acquired knowledge (stability).

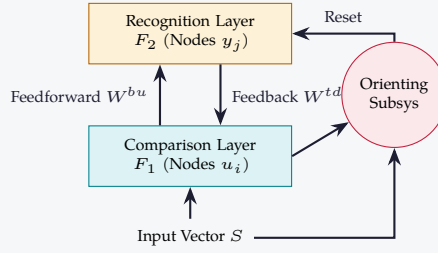
### 2.1 ART1 vs. ART2

- **ART1:** Designed to process binary input vectors ( $\{0, 1\}$ ).
- **ART2:** Designed to process continuous (analog) input vectors.

### 2.2 ART1 Architecture and Operation

ART1 consists of two subsystems: the Attentional Subsystem and the Orienting Subsystem.

## ART1 Network Architecture



## Working Mechanism:

1. **Input presentation:** Input  $S$  is sent to the  $F_1$  layer.
2. **Feedforward activation:**  $F_1$  transmits the pattern to the  $F_2$  layer. The  $F_2$  layer is a competitive layer where the node with the highest net input wins (winner-take-all).
3. **Feedback Template matching:** The winning  $F_2$  neuron projects its prototype vector (template) back down to  $F_1$ .
4. **Vigilance Test:** The orienting subsystem calculates the ratio of match:

$$\frac{|S \cap X|}{|S|} \quad (9)$$

where  $X$  is the activation of the  $F_1$  layer.

- If ratio  $\geq \rho$  (vigilance parameter), **resonance** occurs. The network accepts the match and updates the winning neuron's weights.
- If ratio  $< \rho$ , the orienting subsystem sends a **reset signal** to temporarily disable the winning  $F_2$  node. The search process repeats for other  $F_2$  nodes. If no match is found, a new category node is initialized.

### 3. Boltzmann Machine Learning

The Boltzmann Machine is a recurrent, symmetrically connected network consisting of stochastic binary units (visible and hidden) that operate under statistical thermodynamics principles.

#### Boltzmann Energy & Transition Rules

The total energy  $E$  of a Boltzmann state configuration is defined as:

$$E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i \quad (10)$$

where  $w_{ij}$  is the connection weight,  $s_i \in \{0, 1\}$  is the state of unit  $i$ , and  $\theta_i$  is the threshold bias.

The probability that unit  $i$  transitions to state  $s_i = 1$  is given by the sigmoid-like logistic function:

$$P(s_i = 1) = \frac{1}{1 + e^{-\Delta E_i / T}} \quad (11)$$

where  $\Delta E_i = \sum_j w_{ij} s_j + \theta_i$  is the energy difference and  $T$  is the temperature parameter.

### 3.1 Boltzmann Learning Rule

Boltzmann training occurs in two distinct phases:

1. **Positive (Clamped) Phase:** The visible units are clamped to target training patterns. The network is allowed to reach thermal equilibrium, and the correlation probabilities  $p_{ij}^+$  that units  $i$  and  $j$  are active simultaneously are measured.
2. **Negative (Free-Running) Phase:** No units are clamped. The network runs freely to reach thermal equilibrium, and the free probabilities  $p_{ij}^-$  are measured.
3. **Weight Update:** Weights are updated to minimize Kullback-Leibler divergence:

$$\Delta w_{ij} = \eta(p_{ij}^+ - p_{ij}^-) \quad (12)$$

## 4. Worked Examples

### Backpropagation Step-by-Step Numerical

**Problem:** Consider a simplified neural network containing 1 input node ( $x$ ), 1 hidden node ( $z$ ), and 1 output node ( $y$ ). The transfer function is the binary sigmoid:  $f(net) = 1/(1 + e^{-net})$ .

Given: Input  $x = 1.0$ , target  $t = 0.9$ , learning rate  $\eta = 0.5$ . Initial weights and biases:

- Input-to-hidden weight  $v = 0.5$ , bias  $b_1 = 0.2$
- Hidden-to-output weight  $w = 0.8$ , bias  $b_2 = 0.1$

Calculate the updated weights  $w$  and  $v$  after a single backpropagation epoch.

**Solution:**

**1. Forward Pass:**

- **Hidden Node  $z$ :**

$$net_z = v \cdot x + b_1 = 0.5(1.0) + 0.2 = 0.7$$

$$z = f(net_z) = \frac{1}{1 + e^{-0.7}} \approx 0.668$$

- **Output Node  $y$ :**

$$net_y = w \cdot z + b_2 = 0.8(0.668) + 0.1 \approx 0.634$$

$$y = f(net_y) = \frac{1}{1 + e^{-0.634}} \approx 0.653$$

**2. Backward Pass (Error Propagation):**

- **Output Delta ( $\delta_y$ ):**

$$\begin{aligned} \delta_y &= (t - y)y(1 - y) \\ &= (0.9 - 0.653) \cdot 0.653 \cdot (1 - 0.653) \\ &= 0.247 \cdot 0.653 \cdot 0.347 \approx 0.0560 \end{aligned}$$

- **Hidden Delta ( $\delta_z$ ):**

$$\begin{aligned}\delta_z &= \delta_y \cdot w \cdot z(1 - z) \\ &= 0.0560 \cdot 0.8 \cdot 0.668 \cdot (1 - 0.668) \\ &= 0.0448 \cdot 0.668 \cdot 0.332 \approx 0.00993\end{aligned}$$

### 3. Weight Updates:

- **Hidden-to-Output Weight ( $w$ ):**

$$\begin{aligned}\Delta w &= \eta \cdot \delta_y \cdot z = 0.5 \cdot 0.0560 \cdot 0.668 \approx 0.0187 \\ w(\text{new}) &= 0.8 + 0.0187 = 0.8187\end{aligned}$$

- **Input-to-Hidden Weight ( $v$ ):**

$$\begin{aligned}\Delta v &= \eta \cdot \delta_z \cdot x = 0.5 \cdot 0.00993 \cdot 1.0 \approx 0.00497 \\ v(\text{new}) &= 0.5 + 0.00497 = 0.50497\end{aligned}$$

The updated weights are  $w \approx 0.8187$  and  $v \approx 0.5050$ .

## Quick Revision — Module III

Adaptive Resonance & Backpropagation | PE-EC702C

### Key Formulas at a Glance

|                                                   |                                                                              |
|---------------------------------------------------|------------------------------------------------------------------------------|
| <b>Total Output Error</b>                         | $E = \frac{1}{2} \sum_k (t_k - y_k)^2$                                       |
| <b>Output Layer Delta (<math>\delta_k</math>)</b> | $\delta_k = (t_k - y_k)y_k(1 - y_k)$ (for sigmoid)                           |
| <b>Hidden Layer Delta (<math>\delta_j</math>)</b> | $\delta_j = z_j(1 - z_j) \sum_k \delta_k w_{kj}$ (for sigmoid)               |
| <b>BPNN Weight Updates</b>                        | $\Delta w_{kj} = \eta \delta_k z_j, \quad \Delta v_{ji} = \eta \delta_j x_i$ |
| <b>ART1 Vigilance Ratio</b>                       | $\text{Match} = \frac{ S \cap X }{ S } \geq \rho$ (where $\rho \in [0, 1]$ ) |
| <b>Boltzmann Energy Configuration</b>             | $E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i$                    |

### One-Line Definitions

- **Backpropagation:** A supervised gradient-descent algorithm that updates network weights to minimize the sum of squared errors.
- **Stability-Plasticity Dilemma:** The goal of learning new input patterns without overwriting previously learned representations.
- **Vigilance Parameter ( $\rho$ ):** The threshold parameter in ART that determines how closely an input must match a category template.
- **Resonance:** The state in ART where an input matches a template well enough to initiate weight tuning.
- **Boltzmann Machine:** A recurrent stochastic neural network that uses simulated annealing to escape local energy minima.

### Frequently Asked / Viva Questions

1. **Why is Backpropagation referred to as a "supervised" learning algorithm?**  
Because weight adjustments are calculated using the difference (error) between the network's computed output and the target (label) vector.
2. **State the purpose of the activation function's derivative  $f'(net)$  in backpropagation.**  
It scales the weight update based on the slope of the activation curve. In flat regions (near 0 or 1 for sigmoids), updates are very small, preventing oscillations.
3. **What is the effect of setting a high vigilance parameter ( $\rho$ ) in ART1?**  
A high vigilance value (e.g.  $\rho = 0.95$ ) requires highly precise matches, leading the network to create many narrow, fine-grained categories.
4. **What is the effect of setting a low vigilance parameter ( $\rho$ ) in ART1?**

## Frequently Asked / Viva Questions (contd.)

A low vigilance value (e.g.  $\rho = 0.3$ ) allows loose matching, causing the network to group highly different inputs into a few broad, coarse categories.

5. **How does Boltzmann learning utilize the positive and negative phases?**

The positive phase clamps inputs to capture target correlations ( $p_{ij}^+$ ), while the negative phase runs freely to capture network-generated correlations ( $p_{ij}^-$ ).

6. **Why does backpropagation sometimes get stuck in local minima?**

Because gradient descent follows local gradients on the error surface. If it enters a local valley (minimum) that is not the absolute lowest point (global minimum), it cannot escape.

7. **What is the function of the orienting subsystem in ART1?**

It acts as a controller that compares the input match ratio with the vigilance parameter  $\rho$ . If it is too low, it issues a reset signal to disable the winning category node.

8. **Define the term "Simulated Annealing" in Boltzmann machines.**

The process of starting at a high temperature  $T$  to allow stochastic state jumps (escaping local minima), and slowly cooling down to settle in the global minimum state.

9. **What is the difference between feedforward and feedback networks in terms of stability?**

Feedforward networks compute outputs instantaneously and are always stable. Feedback networks have recurrent loop dynamics and can oscillate or require iterative convergence to reach stability.

10. **Why is  $\frac{1}{2}$  added in the error formulation  $E = \frac{1}{2} \sum (t - y)^2$ ?**

It is a mathematical convenience that cancels out the coefficient 2 when taking the derivative  $\frac{\partial E}{\partial y} = -(t - y)$  during gradient calculations.

## Comparison of Adaptive Architectures

| Attribute       | Backpropagation BPNN | ART1                               | Boltzmann Machine                |
|-----------------|----------------------|------------------------------------|----------------------------------|
| Learning Type   | Supervised           | Unsupervised                       | Supervised / Unsupervised        |
| Connections     | Feedforward only     | Bidirectional (top-down/bottom-up) | Symmetric recurrent loops        |
| Neuron States   | Analog / Continuous  | Binary $\{0, 1\}$                  | Stochastic Binary $\{0, 1\}$     |
| Page Resolution | Fixed classes        | Dynamic category creation          | Probability distribution fitting |